

CSCE 606 SOFTWARE ENGINEERING · MODULE 03 · LECTURE NOTES

Modularity: Coupling and Cohesion

The one design idea that actually reduces change cost.

MODULE OUTCOMES

- Define coupling and cohesion and recognise each in code.
- Explain information hiding and what a module should hide.
- Distinguish deep modules from shallow ones.
- Recognise when abstraction adds cost rather than removing it.
- Judge a decomposition by the change it makes cheap.

1 What modularity is for

A MODULE BOUNDARY IS A BET ABOUT CHANGE

Drawing a boundary asserts that what is inside will change independently of what is outside. If that is true, changes stay local and cheap. If it is false, every change crosses the boundary and you have added ceremony without benefit.

So the question is never whether code is 'well organised' in the abstract. It is: **which change does this structure make cheap, and is that the change that will actually come?**

This also means a decomposition that was correct can become incorrect. When the axis of change shifts — a new platform, a new regulation, a new scale — the boundaries that absorbed the old changes may cut directly across the new ones. That is not a failure of the original design; it is information arriving.

1.1 Coupling and cohesion

	Coupling	Cohesion
Scope	Between modules.	Within a module.
Measures	How much one module must know about another's internals.	How closely the parts of a module belong together.
Want	Low.	High.
Symptom when wrong	A change here forces a change there.	The module has no single reason to exist, so every change touches it.

	Coupling	Cohesion
Practical test	To change A, how much of B must I read?	Can I describe what this module does in one sentence, with no 'and'?

Coupling type	Description	Verdict
Data	Modules communicate by passing simple values.	Best available.
Stamp	A whole record is passed where only part is used.	Acceptable. Mildly over-shares, which matters if the record changes.
Control	One module passes a flag that selects the other's behaviour.	Suspicious. The caller now knows about an internal decision.
Common	Modules share global mutable state.	Bad. Any module can change what any other sees, so reasoning is no longer local.
Content	One module reaches into another's internal representation.	Worst. The boundary exists only on paper.

THE BOOLEAN PARAMETER SMELL

`process(data, validate=True)` requires the caller to know what `validate` changes — which is coupling to an internal decision, and the flag's meaning must be rediscovered at every call site.

Two functions are usually better: `process(data)` and `process_unvalidated(data)`. Each has one behaviour, one name, and one documented contract.

The exception is real: a flag configuring genuinely orthogonal behaviour, where splitting produces a combinatorial explosion of function names. This is judgement, not a rule — but a flag that makes the function do something *different*, rather than doing the same thing differently, is almost always two functions.

2 Information hiding

David Parnas's 1972 paper *On the Criteria To Be Used in Decomposing Systems into Modules* is the most important piece of writing about software structure, and the argument is simple enough to state in a paragraph.

The obvious decomposition follows the **processing sequence**: a module per step of the flowchart. Parnas proposed decomposing instead so that each module **hides a design decision that is likely to change**. The interface exposes what callers genuinely need; the implementation conceals the decision.

```
BY PROCESSING STEP:
  read_input -> tokenize -> count -> sort -> print

"Change how word counts are stored."
-> count, sort, and print all break.  THREE modules change.

BY HIDDEN DECISION:
  InputSource  hides  where input comes from
  WordStore    hides  how counts are represented
  Formatter    hides  how output is arranged

"Change how word counts are stored."
-> WordStore changes.  ONE module.
```

Same program, same functionality, radically different change cost. The test for a boundary is therefore: *name a plausible change, and count the modules it touches*. One is the target.

3 Deep and shallow modules

John Ousterhout's framing in *A Philosophy of Software Design* gives a measurable version of 'good abstraction': compare the size of the interface against the size of what it hides.

	Deep module	Shallow module
Shape	Small interface, substantial implementation.	Large interface, thin implementation.
Example	A file system: open, read, write, close, seek — five calls concealing allocation, caching, journaling, and device drivers.	A class that wraps a single library call, renames its parameters, and forwards.
Value	Each interface element learned buys a great deal of capability.	Each element learned buys nothing; you must still understand what is underneath.
Verdict	What to aim for.	A candidate for removal.

MORE ABSTRACTION IS NOT AUTOMATICALLY BETTER

Every layer imposes a cost: it must be learned, navigated during debugging, and kept consistent with what it wraps.

A layer is worth its cost only if it hides more complexity than it introduces. A class that renames one function and calls it has negative value — it adds a file, a name, and a hop, and conceals nothing.

Ousterhout calls the reflexive multiplication of tiny classes **classitis**. Its symptom is that understanding one operation requires opening nine files, none of which contains any logic.

The right question is not 'is this abstracted?' but '**does this abstraction hide more than it costs?**'

An abstraction pays when it...	and does not when it...
Hides a decision that is genuinely likely to change.	Wraps something that has been stable for a decade.
Is used from many call sites.	Has exactly one caller.
Has an interface much smaller than its implementation.	Has an interface as large as its implementation.
Eliminates duplicated <i>knowledge</i> .	Eliminates duplicated <i>text</i> that is coincidentally similar.
Lets a reader reason locally without looking inside.	Must be read through anyway to understand the caller.

3.1 DRY, correctly understood

DRY IS ABOUT KNOWLEDGE, NOT CHARACTERS

The original formulation is that *every piece of knowledge should have a single authoritative representation in the system*. It is a claim about knowledge, not about textual similarity.

Two functions that happen to contain the same five lines, for unrelated reasons, are not duplication. Merging them couples two things that will later need to diverge — and when they do, the merged function grows a flag to serve both callers, which is control coupling arriving by the front door.

The test: if the requirement changes, must both copies change? If yes, it is genuine duplication and should be unified. If no, the similarity is coincidence and unifying it creates a dependency that did not exist.

Premature deduplication is at least as common and at least as expensive as duplication, and it is harder to undo.

4 Assessing a decomposition

- 1 **Name the likely changes** — three to five specific ones — and trace how many modules each touches. More than one is evidence the boundary is in the wrong place.
- 2 **Apply the one-sentence test.** Describe each module's purpose in a sentence without using 'and'. Failure indicates low cohesion: the module is two modules.
- 3 **Draw the dependency graph.** Cycles are a hard signal that two modules are really one. A module that everything depends on is a change bottleneck.
- 4 **Compare interface size against implementation size** for each module. Shallow ones are candidates for inlining.
- 5 **Look for shotgun surgery** — a category of change that reliably touches the same seven files. That pattern names a module that does not exist yet, and is the most dependable symptom of a misplaced boundary.

Note that items 1 and 5 are the same test from opposite directions: predict the change and count the files, or observe the files and infer the missing boundary. The second is more reliable, because it uses evidence rather than speculation — which is an argument for refactoring boundaries after you have seen the changes rather than guessing them in advance.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Parnas — On the Criteria To Be Used in Decomposing Systems into Modules (1972, free)	Ten pages, fifty years old, and still the best thing written on this. Read it in full.

Resource	What it covers for this module
Ousterhout — A Philosophy of Software Design	Deep modules, classitis, and complexity as a measurable property. Short and opinionated; the lecture video is free.
MIT 6.031 — Abstract Data Types, Abstraction Functions	The formal side: representation invariants and what an abstraction boundary actually guarantees.
Martin Fowler — Refactoring catalogue (code smells)	Shotgun surgery, feature envy, and the rest, with the corresponding refactorings.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a program of your own and list five plausible changes. For each, count the files that would need modifying. Identify the worst case and propose a boundary that would fix it.
- 2 Apply the one-sentence test to every module in a codebase you know. Record which fail and what the second responsibility is.
- 3 Do Parnas's exercise: take a small program structured by processing step and restructure it by hidden decision. Measure the change cost before and after on a specific modification.
- 4 Draw the dependency graph for a project. Find the cycles, and the module with the highest fan-in. Propose a change to each.
- 5 Find a shallow module in a real codebase — interface roughly the size of its implementation — and inline it. Judge whether the result is clearer.
- 6 Find two pieces of similar-looking code and determine whether they are genuine duplication by the requirement-change test. If they are not, write down what would have gone wrong had you merged them.
- 7 Use version control history to find shotgun surgery: files that are frequently modified together. git log with some scripting will do it. What missing module does the pattern name?

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is a module boundary a bet about, and how do you judge one?
- 2 Define coupling and cohesion, and give a practical test for each.
- 3 Why is a boolean parameter that selects behaviour usually a smell, and what is the exception?
- 4 State Parnas's criterion for decomposition and contrast it with the obvious alternative.
- 5 What makes a module deep, and what is wrong with a shallow one?

- 6 What does DRY actually claim, and what is the test for genuine duplication?
- 7 What is shotgun surgery and what does it indicate?